

AVL 树、Splay 树与摊还分析

1.1 引言

第一节课将介绍 AVL 树与 Splay 树。作为第一节课，个人认为简单回顾上一学期的内容以引入本学期的内容是非常重要的。在课程简介中已经提到，数据结构的重要性在于可以通过有效管理数据，减少算法中对输入数据或中间数据的搜索、插入、删除、合并等操作的时间。因此，学习数据结构的基本要点就是了解其能支持的操作以及对时间复杂度的改进，从而可以在应用中面对合适的场景选择正确的数据结构。

在数据结构基础中，我们主要学习了如下数据结构以及它们对应的操作的时间复杂度分析，也学习了它们适用的场景：

1. 栈：实现常数级头部添加或删除对象，可用于深度优先搜索；
2. 队列：实现常数级尾部添加和头部删除对象，可用于广度优先搜索。

栈和队列称为线性结构（链表太简单不再重复），以上提到的关键例子（深搜和广搜）只需要用到首尾插入和删除的功能，因此简单的栈与队列是合适的选择。但在数据存储的场景中，数据的更新没有数据的搜索频繁，因此我们更重视搜索的效率，而线性结构往往需要线性时间完成这一点，除非使用有序数组可以用二分查找降低复杂度，但这样插入和删除的操作又变为线性。我们希望搜索、插入、删除的效率都比线性好，那么如何实现呢？

1. 二叉搜索树：遵循左小右大的原则进行插入，因此可以实现在树结构上的“二分”查找。设树高为 h ，则搜索、插入和删除都可以在 $O(h)$ 时间完成（还记得如何实现吗？），如果树形保持平衡， $h = O(\log n)$ ，则实现了我们的理想。
2. 优先队列（二叉堆）：为经常要选取最小值或最大值的算法服务，例如操作系统进程调度，Dijkstra 算法（很多贪心算法都是如此）等。它并不支持高效的任意查找，因为只要求结点的儿子一定要比自己大（或小）即可，但可以支持 $O(1)$ 的 FindMin 或 FindMax， $O(\log n)$ 的插入（还记得吗？可以用数组实现，直接插入在最后一个位置，然后 percolate up）和 DeleteMin（用最后一个元素覆盖根结点，然后 percolate down）。

除此之外，为了支持更快的查询，我们介绍了散列表（hash table）的概念，可以在散列函数满足相应假设时实现期望 $O(1)$ 的查询、插入和删除，但最坏情况下，查询和删除仍可能需要线性时间。事实上前面的例子都告诉我们，要想实现一些特定功能的加速，那必然需要施加一些约束，这些约束可能不好实现，增加设计的复杂程度，也可能会影响其它操作的速度，这其中的 trade off 也使得数据结构的研究充满挑战和趣味。

本讲将介绍平衡搜索树，其设计目的是非常明确的：因为一般的二叉搜索树查询、插入和删除都可以在 $O(h)$ 时间完成（树高为 h ），但存在最差的情况使得 h 和 n 较为接近，这导致搜索树所有操

作效率的降低。因此最简单的想法就是，在二叉搜索树左小右大的大小要求之上，再添加一些有关于树的平衡的要求，使得 $h = O(\log n)$ 总是成立的，便可以真正实现搜索、插入和删除速度的提升。

在开始正式内容之前推荐一个[数据结构可视化网站](#)，包括了我们讲到的所有数据结构的基本操作可视化功能。此外，需要提醒的是，本讲统一把树高定义为根到最深叶子的边数，空树高为 -1 ，单结点树高为 0 ，结点深度是根到该结点的边数，并且以下 $\log$ 均以 2 为底。

1.2 AVL 树

为了设计平衡搜索树，最简单的想法便是强制要求每次插入、删除之后都保证树的平衡（即左右子树高度差较小），这就是 AVL（Adelson-Velskii-Landis）树的思想。这里的平衡并不要求左右子树一模一样，只要求每个结点的左右子树高度差的绝对值不超过 1 。

定义 1.1. 二叉搜索树中，记结点 v 的平衡因子（balance factor）为

$$\text{bf}(v) = h(v.\text{left}) - h(v.\text{right}).$$

若每个结点的平衡因子都属于 $\{-1, 0, 1\}$ ，则称这棵树为 AVL 树。空树也是 AVL 树。

1.2.1 AVL 树的高度

根据如上定义，我们不难证明 AVL 树的高度是对数级别的。

引理 1.2. 含 $n \geq 1$ 个结点的 AVL 树，高度为 $O(\log(n+1))$ 。

证明. 最直接的想法是考虑固定结点数中最高的树的高度，但由于树形太多，难以下手。因此我们可以反向思考：固定高度 h ，AVL 树最少有多少个结点。记这个数为 $N(h)$ 。要让根的高度为 h ，至少一棵子树高 $h-1$ ；为了满足平衡条件，另一棵至少高 $h-2$ 。要使结点最少，两棵子树也分别取自高度下最小的 AVL 树，因此

$$N(-1) = 0, \quad N(0) = 1, \quad N(h) = 1 + N(h-1) + N(h-2) \quad (h \geq 1).$$

设 $M(h) = N(h) + 1$ ，则 $M(h) = M(h-1) + M(h-2) \geq 2M(h-2)$ 。用 $M(-1) = 1, M(0) = 2$ 作初值归纳，得到 $M(h) \geq 2^{(h+1)/2}$ 。所以

$$n+1 \geq N(h) + 1 \geq 2^{(h+1)/2}, \quad h \leq 2 \log(n+1) - 1.$$

当然也可以根据 $M(h)$ 构成斐波那契数列，由斐波那契数列的通项公式得到结论。 $\square$

AVL 树的搜索与普通二叉搜索树完全一致：比较当前键值，向左或向右走，直到找到目标或遇到空指针。因此搜索需要 $O(\log(n+1))$ 时间。然而插入、删除可能会影响树的平衡，因此还要考虑如何恢复平衡。

1.2.2 旋转与插入

对于插入，首先和普通二叉搜索树一样，需要根据左小右大的原则沿搜索路径找到一个空位置，把新结点接上去。在此基础上，我们分析插入后可能出现的失衡情况，并给出恢复平衡的操作。

在介绍具体的失衡类别以及恢复操作前，有一个性质值得关注：被破坏平衡性的结点只能在新结点到根的路径上，因为只有这些结点的子树高度变化了。因此我们修复的逻辑是：从新结点向上走，找到第一个失衡的结点 z ，先修复以它为根的子树。此外，本节不考虑重复键值的情况，假设插入的键值在树中不存在。

插入的失衡类别

先看图 1.1 中的四棵树。图中双圈标出刚插入的结点， z 标出从它向上遇到的第一个失衡结点。

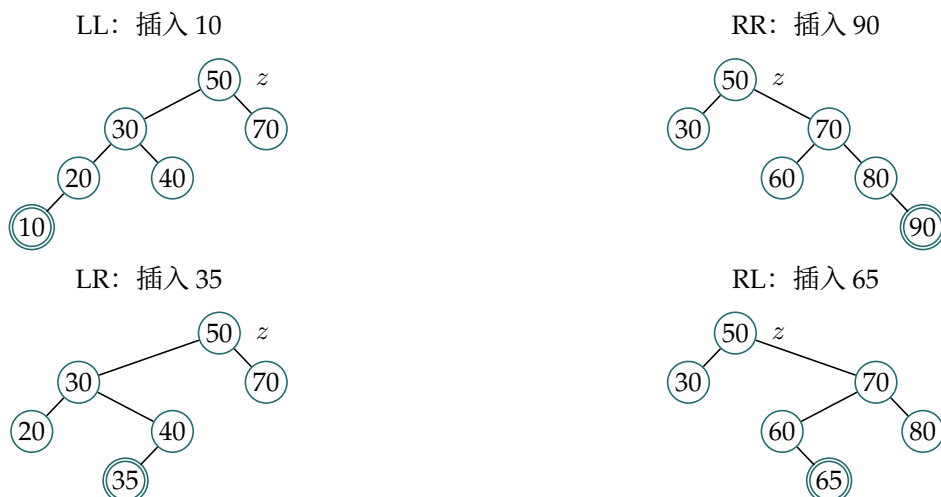


图 1.1 四种插入失衡。双圈为新结点；去掉该结点，即为插入前的 AVL 树。

- **LL**：到新结点 10 的路径是 $50 \rightarrow 30 \rightarrow 20 \rightarrow 10$ 。从 50 出发的前两步都是向左，因此称为 LL。插入前，50 的左右子树高度分别为 1, 0；插入后变成 2, 0，所以 $\text{bf}(50) = 2$ 。
- **RR**：到新结点 90 的路径是 $50 \rightarrow 70 \rightarrow 80 \rightarrow 90$ ，前两步都向右。50 的左右子树高度由 0, 1 变成 0, 2，所以 $\text{bf}(50) = -2$ 。
- **LR**：到新结点 35 的路径是 $50 \rightarrow 30 \rightarrow 40 \rightarrow 35$ ，前两步先左后右。50 的左子树同样由高 1 变成高 2，但这次长高的是左孩子 30 的右子树，故称为 LR。
- **RL**：到新结点 65 的路径是 $50 \rightarrow 70 \rightarrow 60 \rightarrow 65$ ，前两步先右后左。50 的右子树长高，而增加高度的是右孩子 70 的左子树，故称为 RL。

事实上，这四种情况包含了一般的树中由插入引起的所有失衡情况。注意一次插入至多使子树高度增加 1，所以原来平衡的 z 只能变成平衡因子 2 或 -2，并且从平衡被破坏的结点 z 出发，路径的前两条边只有向左、向右两种选择。于是只有 LL、LR、RL、RR 四种组合。

LL 和 RR：一次旋转

对 LL 的例子，50 的左边过高，一种很直观的解决方案就是把左孩子 30 提上来，让 50 成为 30 的右孩子。这一操作称为**右旋** (right rotation)，如图 1.2 上排所示。需要注意的是此时 30 原来的右孩子 40 不能留在原处，因为这个位置要交给 50。由 $30 < 40 < 50$ ，应把 40 改接到 50 的左边，其余结点保持原来的连接。

调整后，新根 30 的平衡因子为 0。不难验证其余结点也都平衡。并且搜索树性质没有改变：中序次序仍为 10, 20, 30, 40, 50, 70 (回忆中序遍历是左根右)。

RR 的例子左右对称：把 50 的右孩子 70 提上来，让 50 成为 70 的左孩子，并将 70 原来的左孩子 60 改接到 50 的右边，其余结点则保留原来的连接。这称为**左旋**（left rotation）。

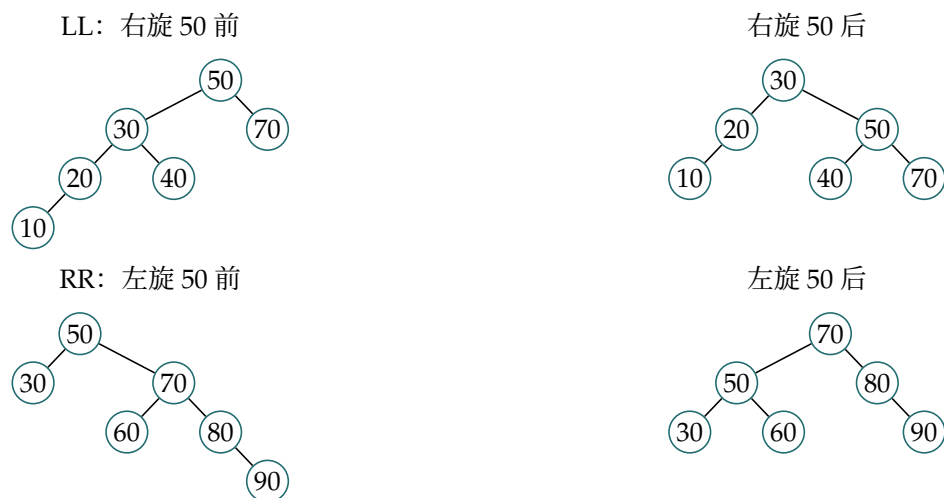


图 1.2 LL 与 RR 的单旋修复。LL 中的 40、RR 中的 60 在旋转时换了父结点。

上面 LL 例中的 40，在旋转时从 30 的右边移到了 50 的左边。如果这里不是一个叶子，而是一整棵子树，只需将它的根接到同样的位置，子树内部不必改变，如图 1.3 所示。不难验证右旋后的树恢复了平衡，且搜索树性质没有改变：（1）由于 z 是从新加入结点出发第一个失衡的结点，并且加入新结点前树是平衡的，因此可以假设 A 的高度为 h ，则 C 的高度为 $h-1$ ，且 B 的高度为 $h-1$ 。因此右旋后 y 的两个孩子高度都是 h ， z 的两个孩子高度都是 $h-1$ ，并且 A, B, C 内部的平衡没有改变，所以旋转后整棵树恢复了平衡；（2）旋转前后的中序次序都是 A, y, B, z, C ，所以搜索树性质没有改变。左旋将左右互换即可，图 1.4 单独画出了 RR 的修复，不难证明其恢复了平衡且保持了搜索树的性质。



图 1.3 右旋与左旋。向右读是对 z 右旋，向左读是对 y 左旋（RR 修复）；方框表示整棵子树，可以为空。



图 1.4 RR 的左旋。中序次序 A, z, B, y, C 保持不变。

LR 和 RL：两次旋转

在 LR 中，50 也是左边过高，因此一个直接的想法就是照搬 LL 的右旋。然而，如果直接右旋，会发现并不能解决问题。若把左孩子 30 提到根，50 成为 30 的右孩子，以 40 为根的子树就必须改接

到 50 的左边。这样新根 30 的左子树只有叶子 20，高 0；右子树却仍含路径 $50 \rightarrow 40 \rightarrow 35$ ，高 2。根的平衡因子变成 -2 ，仍然失衡。

所以我们需要考虑新的旋转方式。图 1.5 中的两步操作可以恢复平衡，其关键在于第一步将 LR 情形转化为 LL 情形。具体过程如下（RL 按相反方向处理，不展开）：

1. 先对 30 左旋，形成可以用右旋修复的 LL 情形；
2. 再对 50 右旋，不难验证旋转后整棵树恢复了平衡，且搜索树性质没有改变。

以上两步都由基本旋转组成，合起来称为**双旋**（double rotation）。

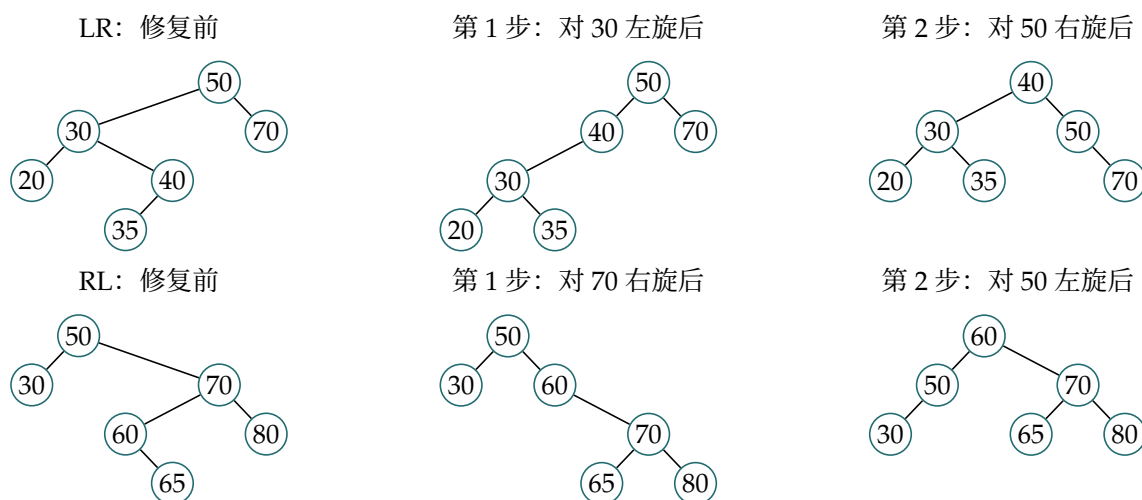


图 1.5 LR 与 RL 的双旋修复。

将双旋的例子推广到一般的子树，如图 1.6 所示，读者可以自行验证旋转后整棵树恢复了平衡，且搜索树性质没有改变。

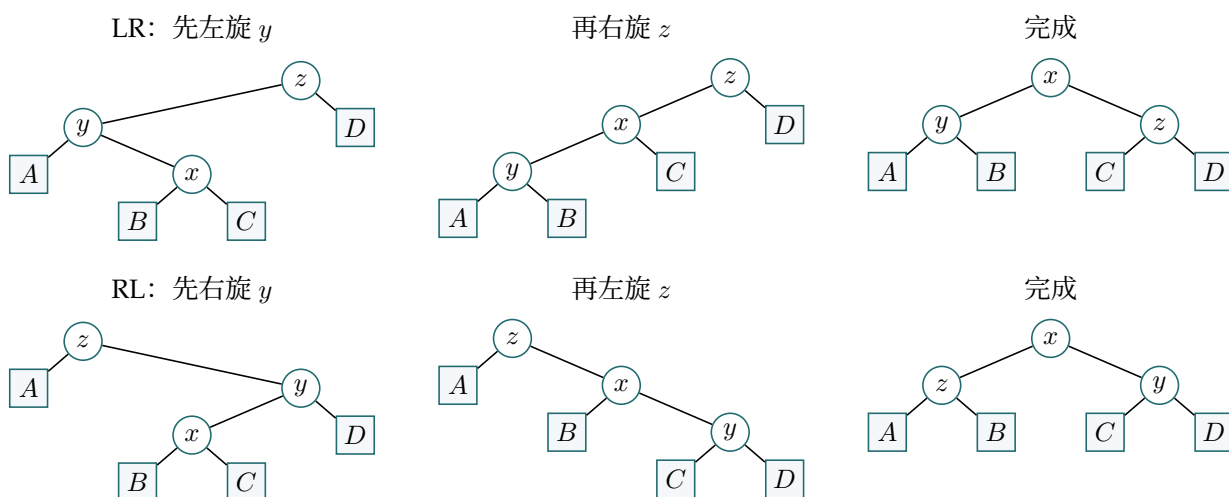


图 1.6 LR 与 RL 的两次旋转。每行从左到右，四棵外围子树各自保持不变。

思考：考虑两个问题：插入会不会同时破坏 AVL 树中多个结点的平衡？如果会，修复最下面一个失衡结点就足够了吗？

这一问题的意义在于，如果一条线上 $O(\log n)$ 个结点平衡性质都被破坏，且需要每个顶点都旋

转的操作代价是比较大的。事实上，两个问题的答案都是肯定的。对于第一个问题，图 1.7 给出了一个例子：插入 10 后，30 和 40 都失衡了。

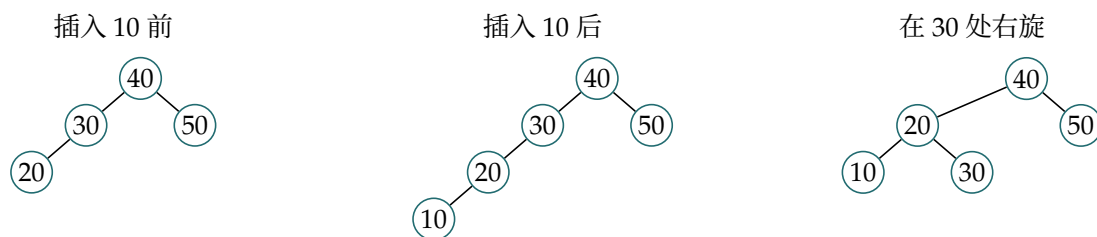


图 1.7 插入 10 使两个结点同时失衡。在 30 处右旋后，30、40 的平衡因子由 2、2 分别变为 0、1。

第二个问题也成立，回忆只有新加入的结点到根结点的路径上的结点可能平衡属性被破坏。因为此时是插入操作，所以平衡被破坏是因为新加的结点太深了。读者不难发现，无论是 LL、RR 还是 LR、RL，在经过旋转恢复后，都会使得 z （第一个失衡的结点）为根的子树高度恢复到插入前的高度。于是对于 z 的父结点而言，它的子树高度也恢复了，因此它的平衡也就恢复了。于是只需修复最下面一个失衡结点即可，因此插入额外引入的旋转次数是常数级别的。

旋转与插入的实现

下面我们给出 AVL 树旋转与插入的伪代码实现，读者可以参考前面的图示来辅助理解。

算法 1.1 AVL 的基本旋转与高度更新

输入：结点含 $key, left, right, height$; $h(\emptyset) = -1$

```

1: function UPDATE( $v$ )
2:    $v.height \leftarrow 1 + \max(h(v.left), h(v.right))$ 
3: end function
4: function RIGHTROTATE( $z$ )  $\triangleright z.left \neq \emptyset$ 
5:    $y \leftarrow z.left; z.left \leftarrow y.right; y.right \leftarrow z$ 
6:   UPDATE( $z$ ); UPDATE( $y$ )
7:   return  $y$ 
8: end function
9: function LEFTROTATE( $z$ )  $\triangleright z.right \neq \emptyset$ 
10:   $y \leftarrow z.right; z.right \leftarrow y.left; y.left \leftarrow z$ 
11:  UPDATE( $z$ ); UPDATE( $y$ )
12:  return  $y$ 
13: end function

```

注意更新高度时先更新 z ，再更新新的根 y ，否则 y 会读到 z 的旧高度。以上函数返回新的根。

算法 1.2 恢复局部平衡与 AVL 插入

```

1: function BALANCE( $z$ )
2:   UPDATE( $z$ )
3:   if  $\text{bf}(z) = 2$  then
4:     if  $\text{bf}(z.\text{left}) < 0$  then
5:        $z.\text{left} \leftarrow \text{LEFTROTATE}(z.\text{left})$ 
6:     end if
7:     return RIGHTROTATE( $z$ )
8:   else if  $\text{bf}(z) = -2$  then
9:     if  $\text{bf}(z.\text{right}) > 0$  then
10:       $z.\text{right} \leftarrow \text{RIGHTROTATE}(z.\text{right})$ 
11:    end if
12:    return LEFTROTATE( $z$ )
13:   end if
14:   return  $z$ 
15: end function
16: function AVLINSERT( $z, k$ )
17:   if  $z = \emptyset$  then
18:     return 新结点 ( $k, \emptyset, \emptyset, 0$ )
19:   end if
20:   if  $k < z.\text{key}$  then
21:      $z.\text{left} \leftarrow \text{AVLINSERT}(z.\text{left}, k)$ 
22:   else if  $k > z.\text{key}$  then
23:      $z.\text{right} \leftarrow \text{AVLINSERT}(z.\text{right}, k)$ 
24:   else
25:     return  $z$ 
26:   end if
27:   return BALANCE( $z$ )
28: end function

```

核心思想是，在插入时首先递归下降找到插入位置，然后回溯更新高度并检查平衡。注意此处我们没有使用父指针，因此在回溯时无法直接访问父结点，只能通过递归返回的方式更新父结点的子树。如果使用父指针，则可以使用循环来实现。

1.2.3 删除简介

本课程并不要求掌握 AVL 树的删除操作，因此这里仅简要拓展。回忆普通搜索树删除分三种情况：叶子直接删除；只有一个孩子时，用孩子替换它；有两个孩子时，找到右子树中最小的结点，用其键值替换当前键值，再到右子树中删除最小结点。最小结点没有左孩子，所以退化到前两种情况。

删除后的旋转仍然使用前面介绍的单旋和双旋，但此时没有插入的元素，因此无法通过此前通向插入结点的路径决定旋转方式。具体而言，从实际移除结点的位置开始向上检查，找到第一个失

衡结点 z ，并将它较高的孩子记为 p 。回忆平衡因子的定义是左子树高度减去右子树高度： $\text{bf}(z) = 2$ 表示左边过高， $\text{bf}(z) = -2$ 表示右边过高。此时再检查 p 的平衡因子，就可以确定旋转方式：

失衡结点	较高孩子的平衡因子	旋转方式
$\text{bf}(z) = 2$	$p = z.\text{left}, \text{bf}(p) \geq 0$	对 z 右旋
$\text{bf}(z) = 2$	$p = z.\text{left}, \text{bf}(p) < 0$	先对 p 左旋，再对 z 右旋 (LR)
$\text{bf}(z) = -2$	$p = z.\text{right}, \text{bf}(p) \leq 0$	对 z 左旋
$\text{bf}(z) = -2$	$p = z.\text{right}, \text{bf}(p) > 0$	先对 p 右旋，再对 z 左旋 (RL)

以 z 的左子树过高为例，我们希望通过右旋将左孩子 p 提上来。如果 p 的左子树更高，那么就前面的 LL 情形一样，可以直接右旋。如果 p 的右子树更高，则需要先对 p 左旋，再对 z 右旋，这与前面的 LR 双旋相同。右子树过高时，左右方向互换。

删除时还可能出现 $\text{bf}(p) = 0$ ，也就是较高孩子的两棵子树等高，这种情况也只需单旋。仍以左边过高为例，设 p 的两棵子树高度均为 h ，那么 p 的高度为 $h + 1$ ，而 z 的右子树高度为 $h - 1$ 。对 z 右旋后， z 的左右子树高度分别为 $h, h - 1$ ，因此它恢复了平衡，高度为 $h + 1$ ；新根 p 的左右子树高度则分别为 $h, h + 1$ ，同样满足平衡条件。

每次检查或旋转后，都要将当前局部子树的高度与本次删除前的高度比较。如果高度没有变化，更高的祖先就不会因为这次删除而失衡，可以停止；如果高度下降，则需要继续检查父结点：这一点与插入情况不同，插入时只需修复最下面一个失衡结点即可。图 1.8 给出了一个例子：删除 14 后，12 先失衡，右旋使它所在的子树高度从 2 降为 1；接着 8 也失衡，需要再做一次右旋。此前思考题中的证明在此处不成立的原因是，旋转后子树高度可能下降，导致父结点的平衡被破坏。

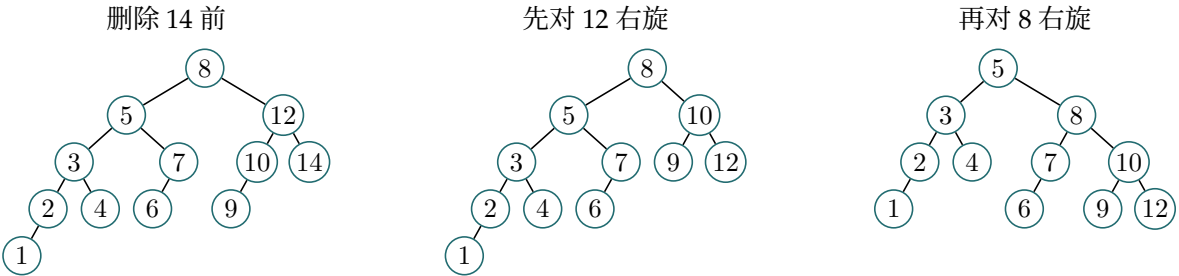


图 1.8 AVL 删除的连续修复。第一棵是合法 AVL 树，删除 14 后依次在 12 和 8 处失衡；最后一棵重新平衡。

最后用一个命题总结 AVL 树的各个操作的时间复杂度。

定理 1.3. 对含 $n \geq 1$ 个结点的 AVL 树，搜索、插入和删除最坏时间均为 $O(\log(n + 1))$ 。一次插入最多两次基本旋转，一次删除最多 $O(\log(n + 1))$ 次基本旋转。

1.3 摊还分析

AVL 树通过严格的平衡约束保证了树高始终为对数级别，从而实现了搜索、插入和删除的对数时间复杂度。接下来要介绍的 Splay 树则希望在放松这一要求的情况下仍然保证某种“平均意义下的”对数时间复杂度，也就是摊还 (amortized) 的对数时间复杂度。本节首先讨论摊还分析的基本概念。

之前的时间复杂度分析都是针对单次操作的最坏情况，然而在实际应用中，算法往往需要执行多次操作。如果一次操作很慢，但这次操作可能改变了结构，使得后续操作变得便宜，那么多次操作的总时间可能并不长。摊还分析的目标就是要估计整串操作的总时间。

定义 1.4 (摊还时间复杂度). 设从初始状态 D_0 开始执行 $m \geq 1$ 次合法操作，每一次操作的实际成本为 $c_1, \dots, c_m$ ；且整个过程中结构的规模均不超过 $N \geq 1$ 。给定函数 $f(N) > 0$ ，若存在常数 $C > 0$ ，使任意这样的初态和操作序列都满足

$$\sum_{i=1}^m c_i \leq Cmf(N),$$

就称在上述初态约定下，每次操作的摊还时间为 $O(f(N))$ 。常数 C 与 m 、 N 、初态和操作序列无关。

因此，“摊还 $O(\log N)$ ”保证的是任意合法序列的总成本为 $O(m \log N)$ ，并不要求每个 c_i 都是 $O(\log N)$ 。不难看出摊还时间复杂度是最坏情况时间复杂度的一个下界，因为最坏情况时间复杂度为 $O(f(N))$ 要求每次操作都满足 $O(f(N))$ ，而摊还时间复杂度只要求合法操作序列下的总成本满足 $O(mf(N))$ 。

注意，摊还时间复杂度并不是平均时间复杂度：平均时间复杂度应当是给出所有连续 m 次操作的可能性时间成本，然后计算所有这些可能性的平均值，因此通常非常难计算，因为每一步都有非常多的选择，连续 m 个操作可能生成的操作路径是指数级别的，并且有时候平均涉及概率分布等，但我们并不知道确切的分布，因此比较难以计算。而摊还时间复杂度则考虑的是连续 m 次操作的总成本上界，因此更容易分析。由此也不难看出，摊还时间复杂度是平均时间复杂度的一个上界。

初态的影响

有时为了方便讨论，摊还分析可能只考虑 D_0 是空结构（例如空树，空堆）的情况，因此这里有必要说明初始状态的影响。事实上，当初始状态非空时，此时证明往往得到

$$\sum_{i=1}^m c_i \leq Cmf(N) + A(D_0),$$

其中 $A(D_0) \geq 0$ 是与初始状态有关、与后续操作次数无关的附加项。注意这一项应当视为总成本上界中为初始状态保留的额度，不一定是程序在开始时初始化的耗时。只有当 $A(D_0) = O(mf(N))$ ，或能用更细的分析消除这一项时，才能推出整个序列平均每次为 $O(f(N))$ ；否则只能得到

$$\frac{1}{m} \sum_{i=1}^m c_i \leq Cf(N) + \frac{A(D_0)}{m}.$$

接下来我们将正式介绍三种摊还分析方法：聚合分析、核算法和势能法。为了便于理解，我们先用一个简单的例子来说明这些方法的基本思想。

1.3.1 聚合分析 (Aggregate Analysis)

接下来首先分析的例子是栈的 MultiPop 操作。假设有一个栈 S ，支持 Push、Pop 和 MultiPop 操作，其中 MultiPop(k) 表示弹出栈顶的 k 个元素（如果栈中元素不足 k 个，则弹出所有元素）。Push 和 Pop 的单次时间成本为 1，MultiPop 的时间成本为 $\min(k, |S|)$ ，其中 $|S|$ 是当前栈的大小。

算法 1.3 MultiPop

输入： 栈 S ；非负整数 k ；Push、Pop 的单次时间为 $O(1)$

```

1: while  $k > 0$  and  $S$  非空 do
2:   POP( $S$ )
3:    $k \leftarrow k - 1$ 
4: end while

```

为了简化分析，我们首先考虑初始状态为空栈的情况。聚合分析的思想非常简单，就是直接根据摊还时间复杂度的定义，计算连续 m 次操作的总成本上界。如果使用最坏情况分析，连续 m 次操作的总成本可能达到 $O(m^2)$ ，因为 MultiPop 可能每次都弹出 $O(m)$ 个元素。然而，实际情况并非如此：不可能从空栈开始连续 m 次操作中每次都弹出 $O(m)$ 个元素，因为每个元素最多只能被弹出一次。

因此我们可以做更精确的分析。从空栈开始，设 m 次操作中有 p 次 Push。注意到每个入栈元素最多被弹出一次，所以所有 Pop 和 MultiPop 加起来最多只能弹出 p 个元素。于是总元素操作数满足

$$C_m \leq p + p \leq 2m.$$

故真实运行时间是 $O(m)$ ，每次操作的摊还时间为 $O(1)$ 。

事实上我们可以做更精确的分析，计算总成本的精确常数。考虑先 Push $m - 1$ 次，最后全部弹出的情况，其成本为 $2m - 2$ 。当 $m \geq 2$ 时，这就是最大的可能成本。因为只要有弹出操作，就至多有 $m - 1$ 次 Push，即 $p \leq m - 1$ ；若完全没有弹出，总成本为 $m \leq 2m - 2$ 。

当然，如果我们不从空栈开始，时间复杂度则需要加上初始栈中元素的数量 n_0 ，因为初始元素带来的最大额外成本就是把它们全部弹出的成本。于是总成本为 $O(m + n_0)$ ，每次操作的摊还时间为 $O(1 + n_0/m)$ 。只要 m 足够大，初始状态的影响就可以忽略。

1.3.2 核算法 (Accounting Method)

不同于聚合分析的直接计算，核算法的思想来源于通过“截长补短”的方式为每一种操作定义新的成本：把耗时长的一部分成本，分摊到耗时短的操作上，使得每种操作的摊还成本都有一个较小的上界。当然，完美的截长补短很难做到，那么我们就需要在尽可能保证总成本变化不大的前提下，把长操作的成本分摊到短操作上。注意，必须保证从成本大的操作截取的部分都要分摊出去，也就是总成本不能变少了，否则算出的总成本反而比实际总成本小，就不能用来证明上界了。

形式化而言，设第 i 次操作的实际成本为 c_i ，分摊后的摊还成本为

$$\hat{c}_i = c_i + \Delta_i.$$

其中 $\Delta_i < 0$ 表示从这次操作截去一部分成本， $\Delta_i > 0$ 表示给这次操作多记一部分成本。为了保证总成本没有少算，要求对任意合法序列都有：

$$\sum_{i=1}^m \hat{c}_i \geq \sum_{i=1}^m c_i, \quad \text{也就是} \quad \sum_{i=1}^m \Delta_i \geq 0.$$

这里允许总成本上升，但不能下降。只要保证总成本没有少算，就可以用 $\sum_{i=1}^m \hat{c}_i$ 来表示真实总成本 $\sum_{i=1}^m c_i$ 的上界，从而得到摊还时间复杂度。并且需要注意的是，对于同一种操作，他们对应的 Δ_i 是相同的，因为核算法是为每种操作定义一个新的成本，而不是为每次操作单独定义。

仍以从空栈开始的 Push、Pop 和 MultiPop 为例。我们要解决的问题是一次 MultiPop 可能弹出很多元素，要把它摊还成本降到常数，就需要把弹出的成本分摊出去。注意到每个被弹出的元素都曾经被压入，所以可以把弹出一个元素的成本，记到当初压入它的那次 Push 上。这样每次 Push 的成本记为 2（即 $\Delta_{\text{Push}} = 1$ ），一次用于入栈，一次用于将来的出栈，而将 Pop 和 MultiPop 的成本都记为 0（即 $\Delta_{\text{Pop}} = -1$ ， $\Delta_{\text{MultiPop}} = -\min\{k, |S|\}$ ）。

一个直观的演示如图 1.9 所示：依次执行 Push(a)、Push(b)、Push(c)、Push(d)、MultiPop(4)。实际成本依次为 1, 1, 1, 1, 4。根据上面的分摊规则，我们可以把最后的 4 分别记到前面的四次 Push 上，便得到 2, 2, 2, 2, 0。这样就实现了“截长补短”，每次操作的摊还成本都不超过 2，并且总成本没有少算。

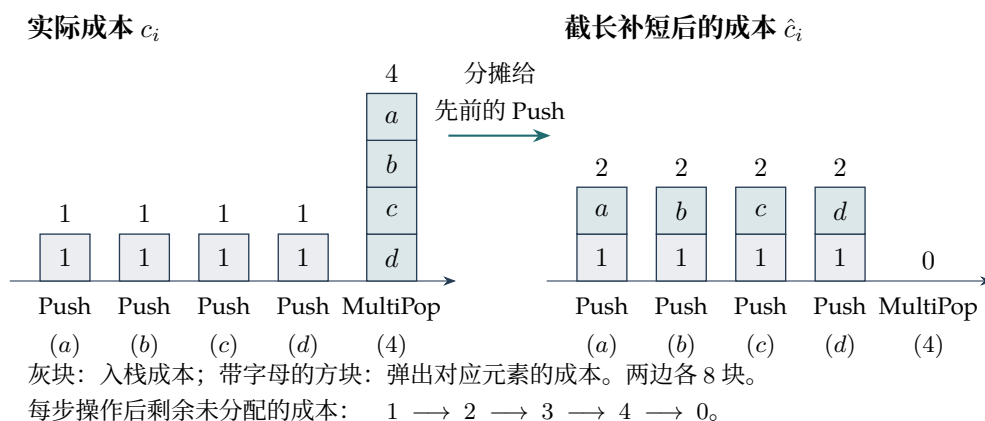


图 1.9 把最后一次 MultiPop 的四单位成本，分别记到压入 a, b, c, d 的四次 Push 上。

事实上，这个分摊办法对任意操作序列都成立。这是因为每次 Push 都给新元素增加了额外 1 单位成本，并且从空栈开始的 Pop 或 MultiPop 的元素都是之前 Push 进来的，因此每次 Pop 或 MultiPop 则只是将这些额外增加的成本用掉而已。根据分摊后的成本，前 m 次操作的摊还总成本至多为 $2m$ ，故实际元素操作总成本也至多为 $2m$ 。从而可以得到每次 $O(1)$ 的摊还时间。若从含 n_0 个元素的非空栈开始，我们可以认为这些元素是 Push 进来的，因此总成本为 $O(m + n_0)$ ，每次操作的摊还时间为 $O(1 + n_0/m)$ 。只要 m 足够大，初始状态的影响就可以忽略。

事实上，核算法这一名字也非常形象。这就像你的银行卡，余额不可以为负数，所以你必须先在花钱之前先存钱，你知道 MultiPop 是个花钱的事情，花钱之前必须存钱。当然你也可以说，你希望你的平均绩点保持在 4.0 以上，你知道 ADS 这门课你可能会挂科，所以你之前的微积分、线性代数、FDS 等就需要为这次挂科存下足够高的均绩。

1.3.3 势能法 (Potential Method)

事实上核算法尽管非常形象，但当结构和操作比较复杂时，你要为每种操作设计一个保证 $\sum_{i=1}^m \Delta_i \geq 0$ 的摊还代价是不一定像上面的例子那么简单的。一个自然的思路是不再把目光局限于每个操作，而是给整个结构定义一个势函数 Φ 用于描述这个结构不同状态，然后规定

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}),$$

因此每一步的摊还代价等于真实操作的代价加上势函数的变化，于是我们求和有

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) = \left(\sum_{i=1}^n c_i \right) + \Phi(D_n) - \Phi(D_0). \quad (1.1)$$

为了使得摊还成本是平均成本的上界, 需要 $\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$, 因此需要 $\Phi(D_n) \geq \Phi(D_0)$, 如果考虑 D_0 为空的情况, 这一点在设计函数的时候很容易满足, 因为可以通过调整使得 $\Phi(D_0) = 0$, 即初始状态势能为 0, 则只需要其它任何状态的势能都不会小于 0 就可以了。

接下来回到空栈开始的 MultiPop 例子。自然地, 我们希望真实操作成本高的 MultiPop 在势能法下的摊还代价回归到常数, 从而达到摊还复杂度 $O(1)$ 的目标。这就要求势能函数在经过 MultiPop 操作后有较大的下降, 抵消 MultiPop 的真实操作代价。事实上这一定义并不难得到: 由于在 MultiPop 后, 栈中的元素个数下降了很多, 因此一个自然的想法就是将势能函数定义为栈中的元素个数, 即 $\Phi(S) = |S|$ 。并且注意这一定义符合 $\Phi(D_n) \geq \Phi(D_0)$, 因此是合理的。

下面需要验证在上述势函数定义下各个操作的摊还代价是否是常数。对于 Push, 真实成本为 1, 势能增加 1, 因此摊还代价为 2; 对于成功的 Pop, 真实成本为 1, 势能下降 1, 因此摊还代价为 0; 对于 MultiPop 弹出 k 个元素, 真实成本为 k , 势能下降 k , 因此摊还代价为 0; 对于空栈 Pop, 真实成本为 0, 势能不变, 因此摊还代价为 0。综上所述, 各个操作的摊还代价都是常数。如果考虑从大小为 n_0 的非空栈开始, 根据式 (1.1), 总成本为

$$\sum_{i=1}^m c_i = \sum_{i=1}^m \hat{c}_i + \Phi(D_0) - \Phi(D_m) \leq \sum_{i=1}^m \hat{c}_i + \Phi(D_0) = O(m + n_0),$$

因此每次操作的摊还时间为 $O(1 + n_0/m)$ 。只要 m 足够大, 初始状态的影响就可以忽略。

事实上, 我们可以从上面的例子中总结一个经验: 为了实现摊还分析得到比较好的复杂度结果, 在核算法和势能法中, 必然需要对真实代价比较高的操作添加一个比较大的负数项, 从而得到一个符合要求的摊还代价。特别地, 在势能法中, 这就要求**势能函数在代价较高的操作后会有较大的下降**, 读者在后续章节的分析以及完成相关习题时都可以回忆这一基本的摊还分析原则。最后, 其实势能法就是借用了物理里面的势能的思想, 给数据结构定义一个与结构本身特点相关的量。比如重力势能是质量和高度的函数, 但树的势能可能是树高的函数, 也可能是树的结点总数的函数甚至更复杂等等。

1.3.4 综合应用

在理解了摊还分析的基本方法后, 我们来看几个例子, 进一步理解这些方法 (特别是势能法) 的应用。

例题 1.5 (二进制计数器). 用一个 k 位的二进制计数器表示非负整数, 初始值为 0, 每次对计数器执行加一操作。证明在连续执行 m 次加一操作后, 总的翻转位数不超过 $2m$ 。

解答. 这一问题与后续章节中二项堆的连续插入有密切联系。这一题目的难点在于, 单次加一操作的最坏情况可能需要翻转很多位, 例如从 $0111 \dots 1$ 到 $1000 \dots 0$ 。然而我们要证明, 连续执行 m 次加一操作后, 总的翻转位数不超过 $2m$ 。

我们用势能法来分析这一问题。回忆势能函数构造的原则: 势能函数在代价较高的操作后需要有较大的下降。对于二进制计数器来说, 代价较高的操作是翻转连续的 1 为 0 的操作。因此一种非常自然的势能函数是计数器中 1 的个数。因为每次加一操作, 若末尾有 t 个连续的 1, 则需要翻转这 t 个 1 为 0, 并将末尾的 0 翻转为 1。翻转的总位数为 $t + 1$, 而势能下降了 $t - 1$, 因此摊还成本为 $(t + 1) - (t - 1) = 2$ 。如果末尾没有连续的 1, 则只需翻转一个位, 势能加 1, 摊还成本为 2。因此每

次加一操作的摊还成本都不超过 2。此外，0 的初始势能为 0。因此连续执行 m 次加一操作后，总的翻转位数不超过 $2m$ 。

当然，这一例子也可以用聚合分析和核算法来分析。聚合分析可以直接找出各位翻转的规律，再求和。把最低位记为第 0 位：它每次加一都翻转；第 1 位每两次加一翻转一次；第 2 位每四次加一翻转一次。一般地，第 j 位只有在低 j 位原来全为 1、向它产生进位时才翻转，因此从全 0 开始，它恰好第 $2^j, 2 \cdot 2^j, 3 \cdot 2^j, \dots$ 次加一时翻转。所以前 m 次加一中，第 j 位恰好翻转 $\lfloor m/2^j \rfloor$ 次，因此总翻转位数为

$$T(m) = \sum_{j=0}^{k-1} \left\lfloor \frac{m}{2^j} \right\rfloor \leq m \sum_{j=0}^{k-1} \frac{1}{2^j} = 2m(1 - 2^{-k}) \leq 2m.$$

如果使用核算法，我们需要将代价高的将 1 翻转至 0 的操作的成本分摊到之前的将 0 翻转至 1 的操作上：每次将 0 翻转为 1 的操作支付 2：1 用于翻转，1 留给将来的复位。由于所有的 1 都是由之前的 0 翻转而来，因此每次将 1 翻转为 0 的操作都可以从之前的翻转中支付。这样，每次加一操作的摊还成本都不超过 2，总的翻转位数不超过 $2m$ 。□

例题 1.6 (双栈实现队列). 用两个初始为空的栈 S_A, S_B 实现队列。入队时把 x 压入 S_A 。出队时，若 S_B 非空则弹出它的栈顶；否则先把 S_A 中元素逐个弹出并压入 S_B ，再弹出 S_B 栈顶。空队列出队则返回失败。每次基本 Push、Pop 操作的成本记为 1，如下哪个势函数能让入队、出队各自的摊还成本都有常数上界？

- A. $2|S_A|$ B. $|S_A|$ C. $2|S_B|$ D. $|S_B|$

解答. 显然，这里代价高的操作是出队。根据势能函数的设计原则，势能函数应当在代价高的操作后有较大的下降，而出队时 S_A 的元素会被搬运到 S_B ，即 $|S_A|$ 会下降很多，因此势能函数应当与 $|S_A|$ 相关。然后由于出队时需要首先将 S_A 的元素 Pop 出，然后再 Push 进 S_B ，因此每个元素的搬运成本为 2，因此应当取 $\Phi = 2|S_A|$ 。这一势函数保证了初始状态势能为 0，且势能始终非负。并且在这一势函数下，入队、出队的摊还成本如下：

- 入队：实际成本 1，势能增加 2，摊还成本 3。
- S_B 非空时出队：实际成本 1， S_A 不变，摊还成本 1。
- S_B 空时出队：实际成本 $2|S_A| + 1$ ，势能下降 $2|S_A|$ ，摊还成本 1。

因此答案为 A。其余选项读者均可验证不符合要求。□

例题 1.7 (SumList). 链表 L 支持两种操作：Insert(x, L) 在表头插入 x ，成本为 1；Sum(L) 把所有元素求和，再用得到的和替换整张表，成本为操作前的长度 $|L|$ 。假设初始为空，并且空表 Sum 后保持为空。判断以下哪一项能证明常数摊还成本：

- A. 核算法：Insert 成本为 2，Sum 成本为 0
 B. 势能法： $\Phi(L) = |L|$
 C. 势能法： $\Phi(L) = -|L|$
 D. 两种方法都不能证明

解答. 显然, 这里代价高的操作是 Sum, 所以我们希望 Sum 时势能下降较多, 因此自然取 $\Phi(L) = |L|$ 。读者不难验证这一选项符合要求, 因此 B 正确, D 错误。对于 C, 读者也不难验证其不符合要求。

A 的问题在于, Sum 并不会把表清空, 而是会留下一个结点。未来再调用 Sum 时, 仍然要处理这个结点, 仍然需要花费 1。例如, 从空表开始依次执行 $\text{Insert}(1, L)$ 、 $\text{Sum}(L)$ 、 $\text{Sum}(L)$, 三次操作的实际成本分别为 1, 1, 1, 总共为 3; 但按 A 的规则, 只在 Insert 时计算成本 2, 两次 Sum 都不计算成本, 因此不够支付实际成本。如果把规则改为 Insert 成本为 2、Sum 成本为 1 则是正确的: 每次插入消耗成本 1, 给新结点留下 1; 求和时如果是新插入的结点则用此前留下的成本, 如果是求和得到的结点则用 Sum 的成本 1 即可。□

1.4 Splay 树

在结束了摊还分析的基本方法后, 我们可以进入 Splay 树的介绍。回忆 Splay 树的想法来源于希望可以不像 AVL 那样保持严格的平衡约束, 但也能保证摊还的对数时间复杂度。Splay 树的操作简而言之就是在普通搜索树操作的基础上加上伸展 (splay) 操作。而所谓伸展操作就是通过一系列旋转将访问的结点移动到根结点的位置。因此这一点符合程序的局部性: 刚刚访问的数据, 很可能很快又要用到, 因此在实现缓存和垃圾收集等算法中有一定的应用, 这也是其相较于 AVL 树的优势所在。

我们可以将搜索、插入、删除简要描述如下:

- 搜索: 使用普通二叉搜索树的方法找到目标结点后, 将它伸展到根; 若查找失败, 则伸展搜索路径上最后一个非空结点。
- 插入: 按普通二叉搜索树的方法插入叶子, 再把新结点伸展到根; 遇到重复键则不插入, 而是伸展已有结点。
- 删除: 先查找要删除的结点并将它伸展到根。删除根后, 需要把剩下的左右子树拼接成一棵搜索树, 具体方法见后文。

由此可以看出, Splay 树操作的核心就是旋转, 因此接下来我们将介绍 Splay 树的旋转方法。

1.4.1 失败的 naive 旋转方法

自然地, 最 naive 的想法就是不断用 AVL 树的单旋把目标结点与父结点交换位置直到根。这个方法称为 move-to-root。它确实把刚访问的结点放到了根, 但是否能保证一长串操作都足够快呢?

考虑如下情况。从空树开始依次插入 $1, 2, \dots, N$, 每次插入后都将插入的结点单旋到根, 最后得到的树以 N 为根, N 的左孩子是 $N-1$, $N-1$ 的左孩子是 $N-2$, 依此类推, 直到结点 1; 所有结点都没有右孩子。图 1.10 展示了其中插入 5 的过程。

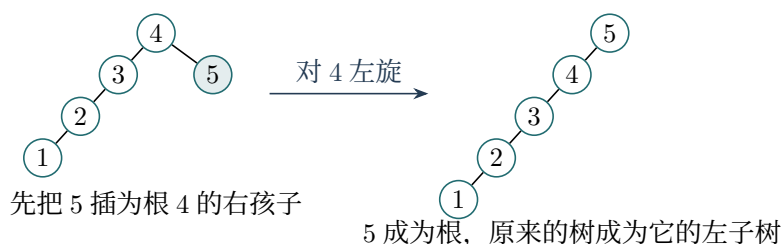


图 1.10 move-to-root 插入 5 的过程: 先按二叉搜索树的规则插入, 再通过一次左旋将 5 移到根。

随后依次查找 $1, 2, \dots, N$ 。查找 1 后, 1 到了根, 右孩子是 N ; 从 N 开始沿左孩子向下走, 依次经过 $N-1, N-2, \dots, 2$ 。查找 2 后, 2 到了根, 左孩子是 1, 右孩子是 N ; 从 N 开始沿左孩子向下走, 依次经过 $N-1, N-2, \dots, 3$ 。图 1.11 展示了查找 1 和 2 的过程。

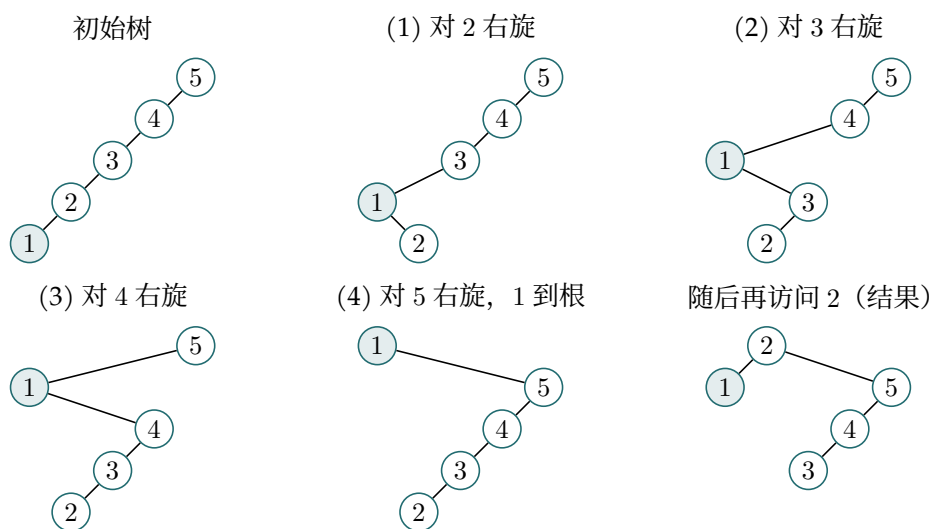


图 1.11 move-to-root 的反例取 $N = 5$, 按从左到右、从上到下的顺序阅读。访问 1 时, 依次对它的父结点 2, 3, 4, 5 做右旋; 1 到根后, 从它的右孩子 5 开始, 沿左孩子依次到达 4, 3, 2。最后一幅给出随后访问 2 的结果。

这个规律可以归纳下去: 访问 j ($2 \leq j \leq N$) 前, 根为 $j-1$, 右孩子是 N ; 未访问的结点 $N, N-1, \dots, j$ 从上到下依次相连, 每个后继结点都是前一个结点的左孩子。因此从 2 到 N 的访问深度依次为 $N-1, N-2, \dots, 1$, 访问 1 的深度为 $N-1$ 。只数旋转就有

$$(N-1) + \sum_{j=2}^N (N-j+1) = \frac{(N-1)(N+2)}{2} = \Theta(N^2).$$

加上之前的 N 次插入, 整个序列也只有 $2N$ 个操作, 却要花二次时间, 因此无法实现摊还 $O(\log N)$ 。这说明 naive 的 move-to-root 方法不够好, 需要更精妙的旋转方法。

1.4.2 zig、zig-zag 与 zig-zig

下面我们给出 Splay 树的旋转方法。记当前目标结点为 x , 其父结点为 p ; 当 p 不是根时, 再将 p 的父结点记为 g , 示意图如图 1.12。旋转方法分为三种情况:

1. 若 p 已经是根, 则做一次旋转把 x 放到根, 这一操作称为 zig, 实际上就是 AVL 树的单旋。
2. 当 p 是 g 的左孩子, 而 x 是 p 的右孩子时, 做对应于 AVL 树 LR 的双旋; 当 p 是 g 的右孩子, 而 x 是 p 的左孩子时, 做对应于 AVL 树 RL 的双旋, 这两种左右对称的操作都称为 zig-zag。
3. 当 p 是 g 的左孩子, 且 x 是 p 的左孩子时, Splay 采用的操作称为 zig-zig。它先对 g 右旋, 将 p 提到 g 原来的位置; 然后对 p 右旋, 将 x 提到这棵子树的根。若 p 是 g 的右孩子, 且 x 是 p 的右孩子, 则依次对 g, p 做左旋。

注意这里与 move-to-root 的区别: 以从 g 到 x 的两条边都向左的情形为例, move-to-root 会先对 p 右旋、再对 g 右旋, 而 zig-zig 的顺序正好相反。两种方法虽然都把 x 提高了两层, 但得到的子树结构不同。

每完成一组 zig-zig 或 zig-zag, x 的深度就减少 2。如果 x 还不是根, 就根据它与新的父结点、祖父结点的关系继续旋转; 当 x 只差一层就能到根时, 再用一次 zig 完成伸展。因此一次伸展至多包含一次 zig, 并且它只能出现在最后。读者可以自己尝试用代码实现这些旋转方法。

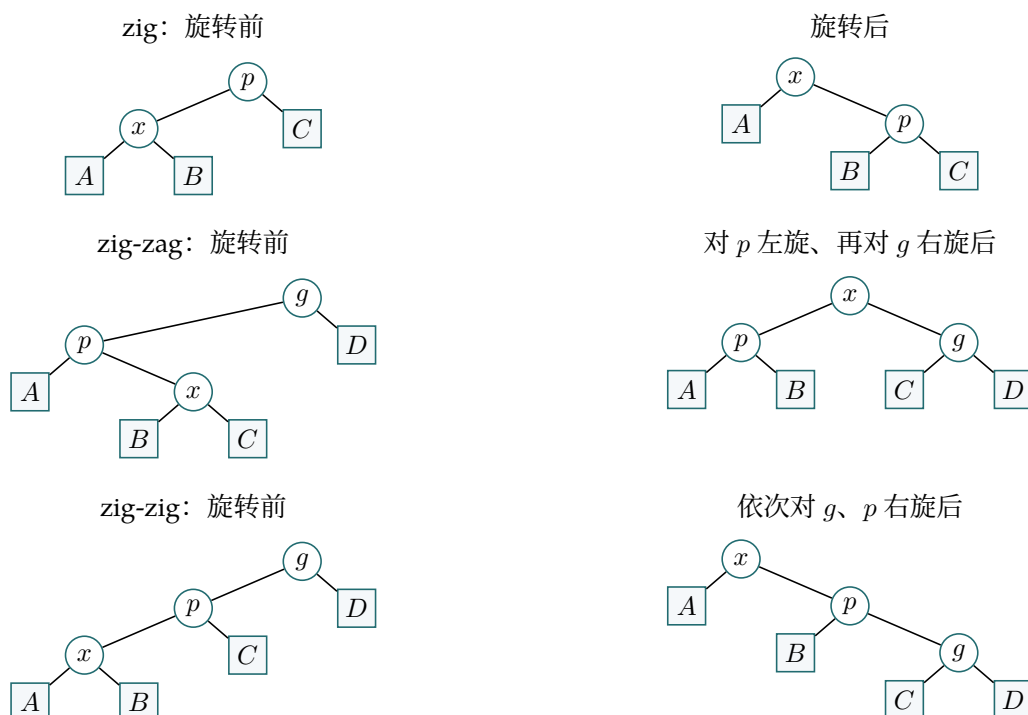


图 1.12 Splay 三类步骤的子树去向 (另一方向为镜像)。zig-zag 与 zig-zig 各做两次基本旋转。

我们来看一个具体的例子: 从空树依次插入 $1, 2, \dots, 7$, 然后访问 1。在插入的过程中, 每次插入后将新结点伸展到根, 最终得到以 7 为根、沿左孩子依次连接到 1 的树 (建议读者自己尝试画出具体步骤)。访问 1 时, 它的父结点是 2, 祖父结点是 3, 因此先对 3 右旋, 再对 2 右旋。完成这一组 zig-zig 后, 1 的父结点变成 4, 祖父结点变成 5, 下一组便依次对 5、4 右旋。最后再依次对 7、6 右旋, 1 就到了根。图 1.13 画出了每组旋转后的树。

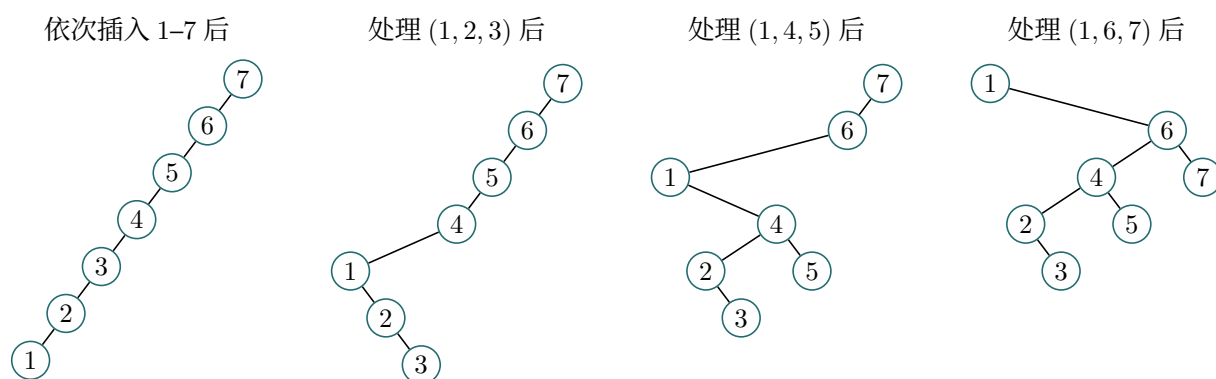


图 1.13 插入 1-7 后访问 1 的完整过程, 共三组 zig-zig、六次基本旋转。

最后讨论删除操作。其中第一步是查找并伸展要删除的结点到根, 没有什么值得讨论的。重点在于删除根结点后的处理。设剩下的左右子树为 L, R 。若 L 为空, 直接以 R 作为删除后的树。否则, 在 L 中找到键值最大的结点 u , 并将它伸展到 L 的根。由于 u 的键值最大, 它不可能有右孩子, 而 R 中的所有键又都大于 u 的键, 因此可以把 R 接为 u 的右子树。这样就把两棵树拼接成了一棵二叉

搜索树。图 1.14 展示了删除结点 4 的过程。

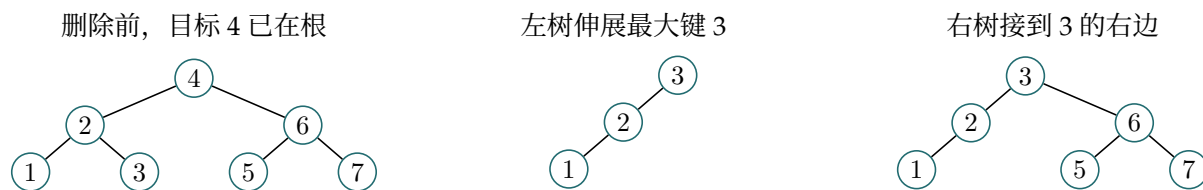


图 1.14 删除 4 后拼接左右子树。中间图只画左树，右树 6(5,7) 暂时独立，最后完整接回。

1.4.3 Splay 树的摊还分析

本节我们通过势能法来分析 Splay 树的摊还时间复杂度。当 Splay 树访问一个很深的结点时，需要沿搜索路径向下查找，再通过多次旋转把它移到根。按照前面构造势函数的思路，我们希望这样的访问能使势能大幅下降，以抵消较高的实际成本。

当然对这一问题而言，没有像之前那么直观的势函数取法，这里直接给出这一巧妙的构造。记 $s(v)$ 为以 v 为根的子树结点数（包含 v 自身），取

$$r(v) = \log s(v), \quad \Phi(T) = \sum_{v \in T} r(v), \quad \Phi(\emptyset) = 0.$$

这样取的好处有两点，首先定义中对数的出现有助于最后得到对数复杂度；其次，一次基本旋转只会改变参与旋转的两个结点的 $s(v)$ ，因此计算一次旋转的势能变化时就只需考虑这两个结点，非常方便。但我们很难解释其他的直观，只能说需要一定的尝试和经验，例如读者可以尝试用树高等方式定义势函数，观察为什么无法得到结论。

我们将 $r(v)$ 称为秩。为了说明这个势函数确实有用，我们先只考虑旋转带来的成本（先忽略查找时间）。设一次伸展共做了 C 次基本旋转，势能变化为 $\Delta\Phi$ ，那么它的摊还成本就是 $C + \Delta\Phi$ 。下面要证明，无论原来的树是什么形状、访问哪个结点，都有

$$C + \Delta\Phi \leq 1 + 3 \log n,$$

其中 n 是树的结点数。这个不等式说明，即使实际旋转次数 C 很大，加上势能变化后也不会超过右边的对数上界。换句话说，超出这个上界的旋转成本，必须由势能下降来抵消。如果旋转次数本来就少，则不一定需要势能下降，只要两者之和满足上界即可。

证明时，我们先分别计算一组 zig、zig-zag、zig-zig 的旋转次数和势能变化，再把一次伸展中的所有旋转加起来。为此，先准备一个之后会用到的对数不等式。

引理 1.8. 若 $a, b > 0$ 且 $a + b \leq c$ ，则 $\log a + \log b \leq 2 \log c - 2$ 。

证明. 由 $(a - b)^2 \geq 0$ ，有 $ab \leq ((a + b)/2)^2 \leq (c/2)^2$ ，然后两边取以 2 为底的对数即可。□

先分析一组旋转

记一组旋转前后的秩分别为 r, r' ，并以基本旋转次数作为这组操作的实际成本。下面结合图 1.12 分别计算三种情况的摊还成本。注意每种情况只需分析图中所示的方向，因为左右互换不会改变证明中用到的子树大小关系。

zig。只有 x, p 的秩改变, 而且 $r'(x) = r(p)$, $r'(p) \leq r(p)$, $r'(x) \geq r(x)$, 所以

$$\begin{aligned}\hat{c} &= 1 + r'(x) + r'(p) - r(x) - r(p) \\ &\leq 1 + r'(x) - r(x) \leq 1 + 3(r'(x) - r(x)).\end{aligned}$$

zig-zag。只有 x, p, g 的秩改变, 且 $r'(x) = r(g)$, $r(p) \geq r(x)$, 得到

$$\begin{aligned}\hat{c} &= 2 + r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g) \\ &= 2 + r'(p) + r'(g) - r(x) - r(p) \\ &\leq 2 + r'(p) + r'(g) - 2r(x).\end{aligned}$$

旋转后 p, g 分别在 x 的两棵子树, 并且两棵子树不相交, 因此 $s'(p) + s'(g) = s'(x) - 1 \leq s'(x)$ 。引理 1.8 给出

$$r'(p) + r'(g) \leq 2r'(x) - 2.$$

代回得到

$$\hat{c} \leq 2(r'(x) - r(x)) \leq 3(r'(x) - r(x)).$$

zig-zig。开头的消项与上一种情况相同, 但旋转后 p 和 g 对应的子树相交, 因此后续证明需要修改。由 $r'(p) \leq r'(x)$ 可得

$$\hat{c} \leq 2 + r'(x) + r'(g) - 2r(x).$$

观察图中的 A, B, C, D : 旋转前 x 的子树是 x, A, B , 旋转后 g 的子树是 g, C, D 。这两个集合互不相交, 且都包含在旋转后 x 的子树中。因此

$$s(x) + s'(g) \leq s'(x), \quad r(x) + r'(g) \leq 2r'(x) - 2.$$

从而

$$\hat{c} \leq 2 + r'(x) + (2r'(x) - 2 - r(x)) - 2r(x) = 3(r'(x) - r(x)).$$

这里用到了 **zig-zig** 的旋转顺序。若改用 **move-to-root**, 原来以 x 为根的子树中的某些结点可能在旋转后落入 g 的子树, 两个集合便可能相交, 所以上面的子树大小不等式不再有保证。

伸展到根

接下来我们把一次伸展中的所有旋转加起来, 这一结论称为访问引理 (Access Lemma)。记 T_{before} 为伸展前的树, T_{after} 为伸展后的树, 我们有:

定理 1.9 (访问引理). 设伸展前的树根为 t , 伸展目标为 x 。若 C 是本次伸展的基本旋转数, 则

$$C + \Phi(T_{\text{after}}) - \Phi(T_{\text{before}}) \leq 1 + 3(r(t) - r(x)) \leq 1 + 3 \log n,$$

右边的 r 取伸展前的值。

证明. 设共做 q 组旋转, 第 j 组结束后目标的秩为 $r_j(x)$ 。每个 **zig-zig** 或 **zig-zag** 的摊还成本至多为 $3(r_j(x) - r_{j-1}(x))$; 至多一个末尾 **zig** 多出常数 1。于是

$$\sum_{j=1}^q \hat{c}_j \leq 1 + 3 \sum_{j=1}^q (r_j(x) - r_{j-1}(x)) = 1 + 3(r_q(x) - r_0(x)).$$

最终 x 是根，子树包含全树，所以 $r_q(x) = \log n = r(t)$ 。 x 本来就是根时，旋转数和势能变化都为零，不等式仍成立。 $\square$

搜索、插入、删除与初始状态的影响

搜索。 在上面的讨论中，我们只考虑了旋转带来的复杂度，而一次完整的搜索还需要从根出发，沿搜索路径找到目标结点 x 。如果 x 在搜索前的深度为 d ，那么查找过程会访问 $d+1$ 个结点，随后的伸展则恰好需要 d 次基本旋转。由于每次比较、旋转以及相应的父指针更新都只需要常数时间，因此可以选取一个固定常数 K ，使整个搜索操作的实际成本 c 不超过 $K(d+1)$ 。

为了把这些成本一起纳入势能分析，我们将势函数乘以同一个常数，记为 $\Psi = K\Phi$ 。根据前面已经证明的访问引理， $d + \Delta\Phi \leq 1 + 3\log n$ ，于是

$$c + \Delta\Psi \leq K(d + 1 + \Delta\Phi) = O(\log(n+1)).$$

以上分析合并了查找与伸展的成本，此处写成 $n+1$ 是将空树的情况也统一进来。

如果查找失败，我们会在搜索路径的末端遇到空指针，然后将最后访问的非空结点伸展到根。与成功查找这个结点相比，失败查找只多了一次空指针判断，因此仍然可以用上面的分析得到对数摊还时间。如果原来的树就是空树，则判断根为空后便结束，实际时间为 $O(1)$ 。

插入。 按照前面介绍的实现方法，我们先将新结点作为叶子插入，再把它伸展到根。查找插入位置以及伸展的部分与上面搜索的分析一致，区别在于插入新结点会改变树的结构，从而影响势能，接下来就要分析这一影响的大小。

设插入前有 n 个结点。新叶子的子树只有它自身，所以它的秩为 $\log 1 = 0$ ，不会直接增加势能。对原来的结点而言，只有新叶子的祖先需要将子树大小从 s 更新为 $s+1$ ，其余结点的子树大小不变。将这些祖先按从下到上的顺序排列，并记它们在插入前的子树大小为 $s_1 < s_2 < \dots < s_d \leq n$ ，则接入叶子但尚未伸展时，势能的增加量为

$$\sum_{j=1}^d \log \frac{s_j + 1}{s_j} \leq \sum_{s=1}^n \log \frac{s+1}{s} = \log(n+1).$$

第一个不等式成立，是因为这些子树大小都是 1 到 n 之间互不相同的整数，而每一项 $\log((s+1)/s)$ 都非负，所以补上其余整数对应的项只会使总和增大。这说明，即使新叶子有很多祖先，接入叶子带来的势能增加仍然不超过 $\log(n+1)$ 。

结合查找与伸展的成本和接入叶子带来的势能增加，可以得到插入的摊还时间为 $O(\log(n+1))$ 。如果插入重复键，不必接入新叶子，只需搜索并伸展已有结点；在空树中插入时，新结点直接成为根，实际时间为 $O(1)$ 。

删除。 删除操作首先查找目标结点并将它伸展到根，这部分摊还成本已经在上面分析过。如果目标不存在，操作到此结束；如果目标存在，则还需要分析删除根以及拼接左右子树带来的势能增加。

删除根以后，我们暂时将两棵子树的势能之和作为当前状态的势能。由于其余结点各自的子树都没有改变，这一步只去掉了原根对势能的贡献 $\log n$ ，所以势能不会增加。接着，若左子树非空，就在其中查找键值最大的结点，并将它伸展到左子树的根。查找与伸展的摊还时间仍可由访问引理得到，为 $O(\log(n+1))$ 。

最后将右子树接到这个新根的右侧。此时只有新根的子树大小发生变化：若左子树原有 $a \geq 1$ 个结点，接上右子树后，新根的子树就包含剩余的全部 $n-1$ 个结点。因此，这一步增加的势能为

$\log(n-1) - \log a \leq \log(n-1)$. 若左子树为空, 则直接将右子树作为删除后的树, 不需要伸展和拼接, 也就没有这部分势能增加。将上述各步相加, 并计入删除结点和连接指针的常数成本, 可得删除操作的摊还时间同样为 $O(\log(n+1))$ 。

综上所述, 从空树开始的 Splay 树的搜索、插入、删除操作都具有对数摊还时间。最后, 如果初始树有 $n_0 \geq 1$ 个结点, 那么每个结点的子树大小至多为 n_0 , 所以初始势能至多为 $n_0 \log n_0$ 。由此我们可以得到以下结论:

定理 1.10. 从空树开始执行 m 次搜索、插入或删除, 若整个过程中的结点数至多为 $N \geq 1$, 则总时间为 $O(m \log(N+1))$ 。若初始树可以是任意含 n_0 个结点的二叉搜索树, 且整个过程中的结点数仍至多为 N , 则上述分析给出的总时间上界为

$$O(m \log(N+1) + n_0 \log(n_0+1)).$$

1.5 习题

习题 1.1 (AVL 树高度). 一棵 AVL 树有 12 个结点, 它的高度是否可能超过 4? 请按本讲以边数定义高度的约定给出理由。如果只要求根的左右子树高度差不超过 1, 而不对其他结点作要求, 是否仍能保证全树高度为对数级别?

习题 1.2 (AVL 树插入). 从空树开始, 依次插入 88, 70, 61, 96, 120, 90, 65。请画出每次旋转前后的局部子树以及最终得到的 AVL 树, 并判断以下关系是否成立: 88 是否为根, 70 是否为 65 的父结点, 65 是否为 88 的左孩子, 96 与 70 是否为兄弟, 61 是否为 70 的父结点。

如果要在程序中查询这些关系, 应当如何实现, 时间复杂度是多少? 若查询涉及的键不存在, 统一回答否。

习题 1.3 (AVL 树删除). 对图 1.15 中的 AVL 树依次删除 12 和 1, 请画出两次删除并恢复平衡后的树。

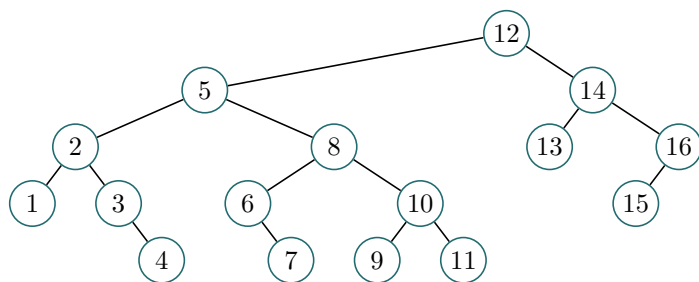


图 1.15 习题 1.3 的初始 AVL 树

习题 1.4 (计数器的初态). 对一个 $k \geq 1$ 位的二进制计数器按模 2^k 执行加一操作, 并以翻转位数作为实际成本。请证明: 从任意初始值开始, 连续 m 次加一的总成本不超过 $2m + k$ 。为什么对于任意的 m , 不能直接省去上界中的 k ?

习题 1.5 (动态数组). 一个动态数组初始为空, 容量为 0。追加元素时, 如果原来的容量 $C > 0$ 且已经用满, 就将容量扩大为 $2C$, 并把旧元素复制到新空间; 如果容量为 0, 则先分配一个元素的位置。假定分配空间本身的成本为 $O(1)$, 每次写入或复制一个元素的成本为 1。

请用聚合分析证明, 从空数组开始追加 m 个元素的总成本为 $O(m)$, 再用势能法分析同一问题。

习题 1.6 (Splay 树与失败查找). 在图 1.13 的最后一棵树上搜索不存在的键 8, 随后删除 4, 画出两次操作后的树。如果每次失败查找后都不做伸展, 是否仍能保证任意操作序列的对数摊还时间?

1.6 参考答案

习题 1.1. 根据前面最少结点数的递推式, $N(4) = 12$, $N(5) = 20$ 。因此, 12 个结点足以构造高度为 4 的 AVL 树, 却不足以构造高度为 5 或更高的 AVL 树, 所以答案是不可能超过 4。

只检查根的平衡因子并不足以保证对数高度。例如, 让根的左右子树各有 h 个结点, 并在每棵子树中把结点沿左孩子逐个相连。两棵子树的高度都是 $h - 1$, 因此根的平衡因子为零, 但全树只有 $2h + 1$ 个结点, 高度却达到 h , 仍然与结点数成正比。□

习题 1.2. 图 1.16 展示了最终得到的 AVL 树, 不难看出五个判断的答案依次为: 是、否、是、否、否。

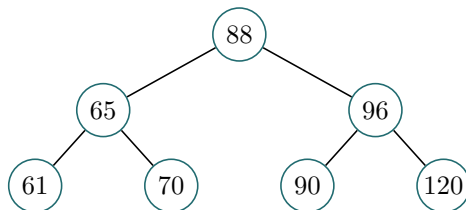


图 1.16 习题 1.2 插入完成后的 AVL 树

建树插入 n 个键的总时间为 $O(n \log(n + 1))$, 单次查询的复杂度显然也是 $O(\log(n + 1))$ 。□

习题 1.3. 删除 12 时, 先用右子树中的最小键 13 替换根的键, 再删除原来位于 14 左侧的叶子 13。此时首先失衡的是 14, 因此需要先对 16 右旋, 再对 14 左旋, 使 15 成为这棵子树的新根。这次双旋使该子树的高度从 2 降为 1, 于是整树的根 13 也失衡了。它的左孩子 5 的平衡因子为零, 对 13 做一次右旋即可恢复平衡。在这棵树上继续删除 1, 结点 2 首先失衡, 对 2 左旋。接着根 5 失衡, 且其右孩子 13 的左子树较高, 因此先对 13 右旋, 再对 5 左旋。两次删除后的结果见图 1.17。

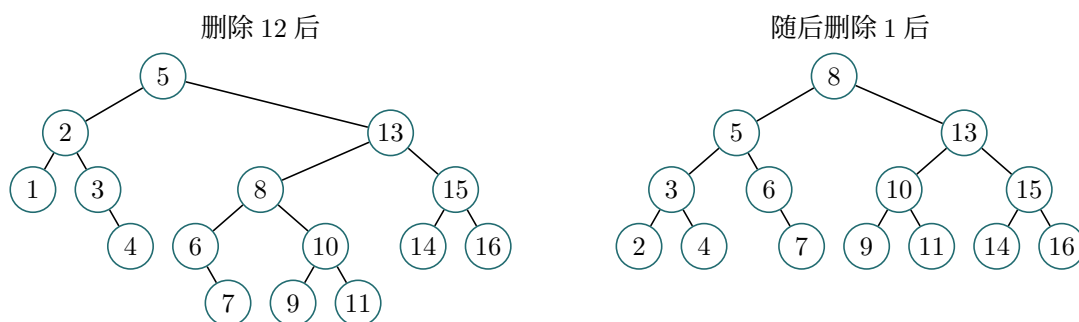


图 1.17 习题 1.3 两次删除并恢复平衡后的 AVL 树

习题 1.4. 初始状态不为 0 引入的额外成本不超过 k , 因为最多有 k 个 1 需要翻转为 0。这也表明即使只执行一次加一, 实际成本也可能达到 $O(k)$, 所以不能省去与初态有关的这一项。□

习题 1.5. 每次追加都需要将新元素写入一次, 所以 m 次追加首先带来 m 次写入。除此之外, 扩容时还需要复制旧元素, 各次复制的元素数依次为 $1, 2, 4, \dots$ 。当 $m \geq 1$ 时, 最后一次复制的元素数小于 m , 因此根据等比数列求和, 所有复制的元素数之和小于 $2m$ 。将写入和复制相加, 总成本小于 $3m$; 若 $m = 0$, 则没有任何操作, 成本为零。

对于势能法, 需要重点考虑扩容时复制大量元素的成本。要使得势函数的下降能抵消扩容时复制带来的大量成本, 势函数中自然需要包含当前容量的相反数一项。然而这不能保证势能非负, 但

注意到扩容后至少一半的空间已有元素，因此可以将势函数定义为 $\Phi = 2n - C$ ，其中 n 是当前元素数， C 是当前容量。不难验证，当不需要扩容时，追加一个元素的实际成本为 1，而势能增加 2，摊还成本为 3；当需要扩容时，复制旧元素的成本为 C ，原先的势能为 $2C - C = C$ ，扩容后势能变为 $2(C+1) - 2C = 2$ ，因此摊还成本为真实成本加上势能变化，即 $C + 1 + (2 - C) = 3$ 。此外，第一次追加需要单独检查：此时分配容量 1，再写入一个元素，元素操作成本为 1，势能从 0 增加到 1，所以摊还成本为 2。综上所述，每种追加操作的摊还成本都有常数上界，再加上每次分配空间的常数成本，总时间为 $O(m)$ 。□

习题 1.6. 搜索 8 的路径为 $1 \rightarrow 6 \rightarrow 7$ ，在 7 的右侧遇到空指针，因此需要将最后访问的结点 7 伸展到根。接下来搜索 4，将 4 伸展到根后删除它。删除根 4 后，左子树最大结点为 3，将其伸展到左子树的根，最后将原来的右子树接为 3 的右子树。三个阶段的结果见图 1.18。

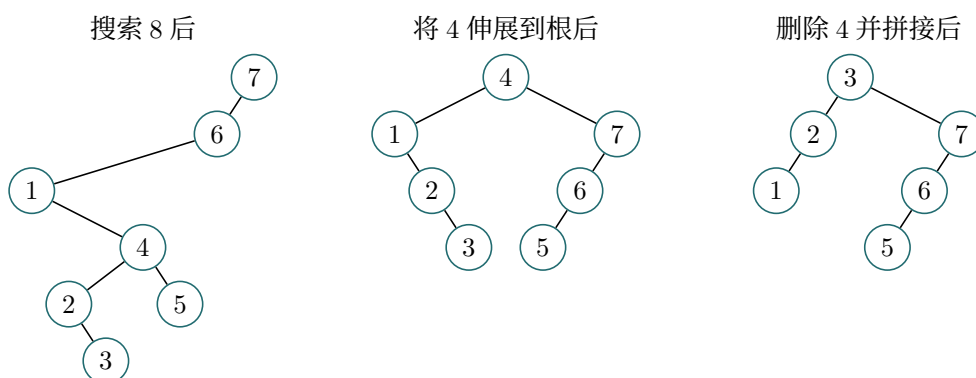


图 1.18 习题 1.6 搜索失败、伸展删除目标及拼接后的 Splay 树

如果失败查找后不做伸展，考虑如下情况：从空树依次插入 $1, \dots, N$ ，得到以 N 为根、沿左孩子依次连接到 1 的树。接着重复搜索 0 共 N 次，每次都要经过全部 N 个结点才会失败，而且搜索后树形不变。仅这些搜索的总时间就达到 $\Theta(N^2)$ ，而连同插入也只有 $2N$ 次操作，因此不能保证对数摊还时间。□

1.7 补充：访问频率与加权势能

在前面的分析中，每个结点对子树大小的贡献都是 1。但实际访问中，有些键可能经常被查找，另一些键却很少被查找。如果这些访问频率差别很大，我们希望得到比统一的对数上界更细的分析，说明经常访问同一批键能带来什么好处。虽然 Splay 算法本身不记录访问频率，但我们可以在证明中为不同的键设置不同的权重。

具体而言，给每个键 v 指定一个固定的正权重 $w(v)$ ，将 $s(v)$ 改为以 v 为根的子树中所有结点的权重之和，仍然取 $r(v) = \log s(v)$ 和 $\Phi(T) = \sum_v r(v)$ 。回顾访问引理的证明，所用到的子树之间的包含关系、互不相交关系以及对数不等式。将结点数换成正权重之和后，这些关系仍然成立；引理 1.8 也只要求 a, b 为正数，并没有要求它们是整数，因此访问引理仍然适用。

设所有结点的总权重为 W 。伸展结束后，目标结点 x 的子树包含整棵树，其秩变为 $\log W$ 。根据访问引理，一次成功访问 x 时，旋转部分的摊还成本至多为

$$1 + 3(r(t) - r(x)) = 1 + 3 \log \frac{W}{s(x)} \leq 1 + 3 \log \frac{W}{w(x)}.$$

这里的 $s(x)$ 取伸展前的值， t 表示整棵树的根，最后一个不等式来自 $s(x) \geq w(x)$ ，因为 x 自身也属于以它为根的子树。

考虑如下情况：设树中有 n 个键，执行 $m \geq 1$ 次成功访问，并记键 v 在这串操作中被访问的次数为 f_v 。为了让访问次数较多的键获得较大的权重，可以取 $w(v) = f_v + 1$ 。这里加 1 是为了让没有被访问的键也有正权重。由于所有访问次数之和为 m ，总权重为 $W = m + n$ 。在这一权重选择下，每棵非空子树的权重至少为 1，所以势能始终非负；每个结点的秩又至多为 $\log(m + n)$ ，所以初始势能至多为 $n \log(m + n)$ 。键 v 对应的访问上界要累加 f_v 次，再将查找时间和初始势能计入，就得到总时间上界

$$O\left(m + \sum_v f_v \log \frac{m + n}{f_v + 1} + n \log(m + n)\right).$$

分析时，我们可以先固定一串访问，再根据其中的次数 f_v 选择权重，因为权重不会改变算法的任何一步操作。因此，这个证明不要求算法提前知道未来的访问次数。

与前面统一按 $\log n$ 估计每次访问相比，这里的上界包含了各个键的访问频率。对于频繁访问的键， $f_v + 1$ 较大，对应的对数项就可能更小。例如，若所有访问都集中在同一个键上，则这一项为 $\log((m + n)/(m + 1))$ ，在 $m \geq n$ 时不超过 1。

这一分析来自 Sleator 与 Tarjan 的 *Self-Adjusting Binary Search Trees* (JACM 32(3), 1985, pp. 652–686)，这也是 Splay 树的经典原始论文。他们还在文中提出了动态最优猜想。在同一个二叉搜索树访问与旋转成本模型下，假设另一个算法事先知道完整的访问序列，并且可以在访问过程中调整树形，那么它完成这串访问所需的最小成本可以作为比较的基准。动态最优猜想所问的是：在允许额外加上一个与初始树规模有关的 $O(n)$ 项的情况下，Splay 处理任意访问序列的总成本，是否都不超过这一最小成本的某个固定常数倍？注意这里的比较对象可以随访问过程改变树形，因此比只与一棵固定的最优搜索树比较更强。事实上这一猜想至今仍是公开问题。

参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第一讲。
2. Mark Allen Weiss, 《数据结构与算法分析：C 语言描述》，第二版。AVL 树与 Splay 树见第 4 章，Splay 的摊还分析见第 11 章。
3. Thomas H. Cormen、Charles E. Leiserson、Ronald L. Rivest、Clifford Stein, 《算法导论》，第三版，第 17 章。
4. Daniel D. Sleator、Robert E. Tarjan, *Self-Adjusting Binary Search Trees*, *Journal of the ACM*, 32(3), 1985, 652–686。本讲的 Splay 访问引理与加权势能分析依据这篇论文；访问引理及其证明见 658–659 页。论文还讨论静态最优性等访问序列性质，并提出动态最优猜想。全文可从[作者页面](#)获取。